Decorator Pattern

The Decorator Pattern is a **structural** design pattern that lets you **add new behaviors** to **objects dynamically** by wrapping them in decorator objects. Instead of modifying the original class or creating subclasses for every combination, you wrap objects in layers that add functionality

Problem Statement

You're building a **Bar Ordering System**. Customers order **drinks** with add add-ons:

Base Spirits:
- Whiskey ($5.00)
- Vodka ($5.00)
- Rum ($4.50)

Add-ons (mixers):
1. Ice - $0.00 (can add multiple for "extra ice")
2. Coke - $1.00
3. Soda - $1.00
4. Lime - $0.50
5. Orange Juice - $1.50

Customers decide add-ons **at the bar**, after the drink is poured. They can add more anytime ("actually, add some lime too").

How do you add behaviors to existing objects dynamically at runtime?

$ 5

Whiskey

5+0 = $5

Whiskey
+
Ice

5+0+1 = $6

Whiskey
+
Ice
+
Coke

5+0+1+1 = $7

Whiskey
+
Ice
+
Coke
+
Soda

Approach 1 (Subclass for every Combination)

```
class Whiskey { }
class WhiskeyWithIce extends Whiskey { }
class WhiskeyWithCoke extends Whiskey { }
class WhiskeyWithIceAndCoke extends Whiskey { }
```

Class explosion -
- With N add-ons, you need 2^N classes for all combinations

Decisions at compile time -
- No Run Time Modifications

Approach 1 (Subclass for every Combination)

```
class Whiskey { }
class WhiskeyWithIce extends Whiskey { }
class WhiskeyWithCoke extends Whiskey { }
class WhiskeyWithIceAndCoke extends Whiskey { }
class WhiskeyWithIceAndLime extends Whiskey { }
class WhiskeyWithIceAndCokeAndLime extends Whiskey { }
class WhiskeyWithIceAndCokeAndLimeAndSoda extends Whiskey { }
```

Class explosion -
- With N add-ons, you need 2^N classes for all combinations

Decisions at compile time -
- No Run Time Modifications

No "extra ice" -
- Would need `WhiskeyWithDoubleIce` , `WhiskeyWithTripleIce`

Approach 2 (Boolean Flags in the Whiskey Class)

```
class Whiskey{
    private boolean hasIce;
    private boolean hasCoke;
    private boolean hasLime;
    private boolean hasSoda;

    public void addIce() { this.hasIce = true; }
    public void addCoke() { this.hasCoke = true; }

    public double getCost() {
        double cost = 5.00;
        if (hasIce) cost += 0.00;
        if (hasCoke) cost += 1.00;
        if (hasLime) cost += 0.50;
        if (hasSoda) cost += 1.00;
        return cost;
    }
}
```

- **Violates OCP** - New mixer = modify class [Mixers are tighlty coupled]
- **No multiples** - Boolean can't represent "extra ice"
- **Doesn't extend** - What about Vodka?

```
Whiskey whiskey = new Whiskey();
whiskey.addIce();
whiskey.addCoke();
```

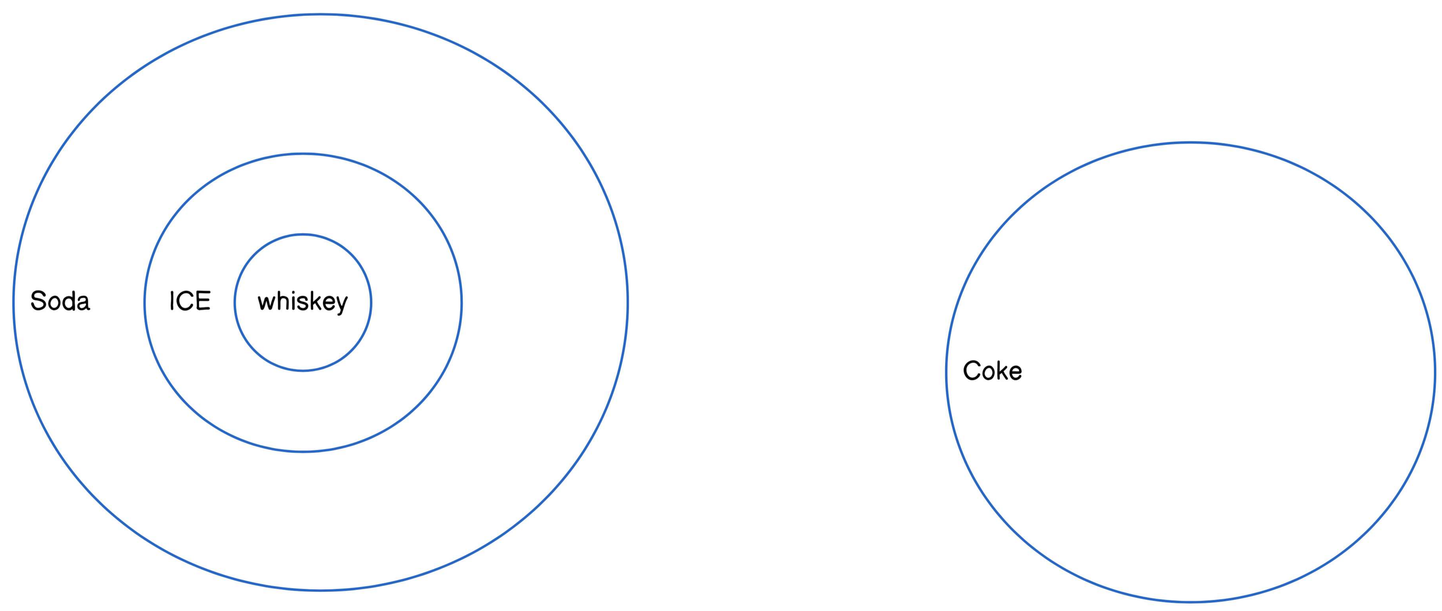
whiskey.getCost()

Decorator Pattern for this problem

add new behaviors to existing objects by wrapping them in

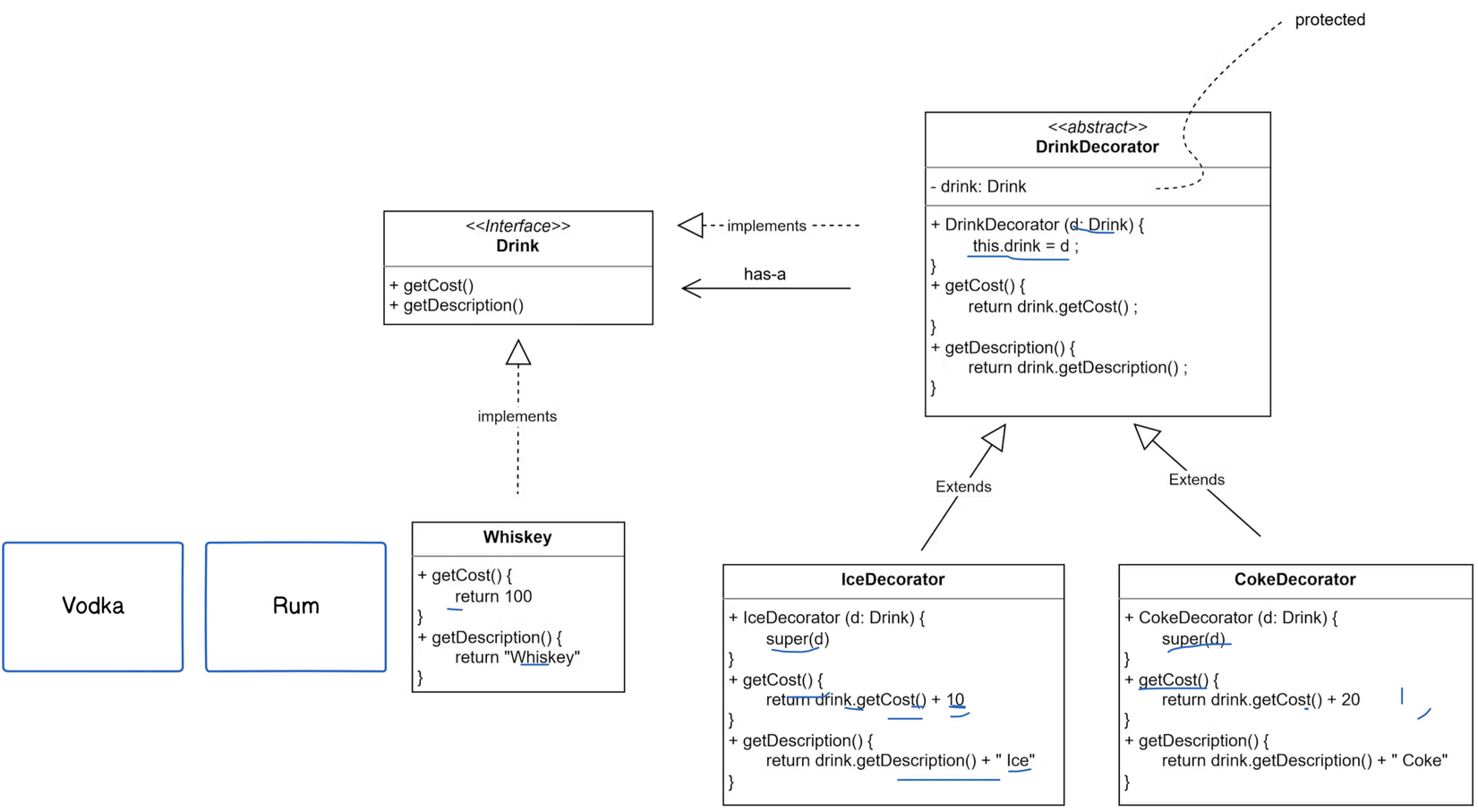
Decorator Pattern for this problem

add new behaviors to existing objects by wrapping them in decorator objects




```
Drink order = new Whiskey(); //base drink
order = new IceDecorator(order);
order = new CokeDecorator(order);
order = new LimeDecorator(order);
```

Decorator Pattern Drink Example Pseudo Code

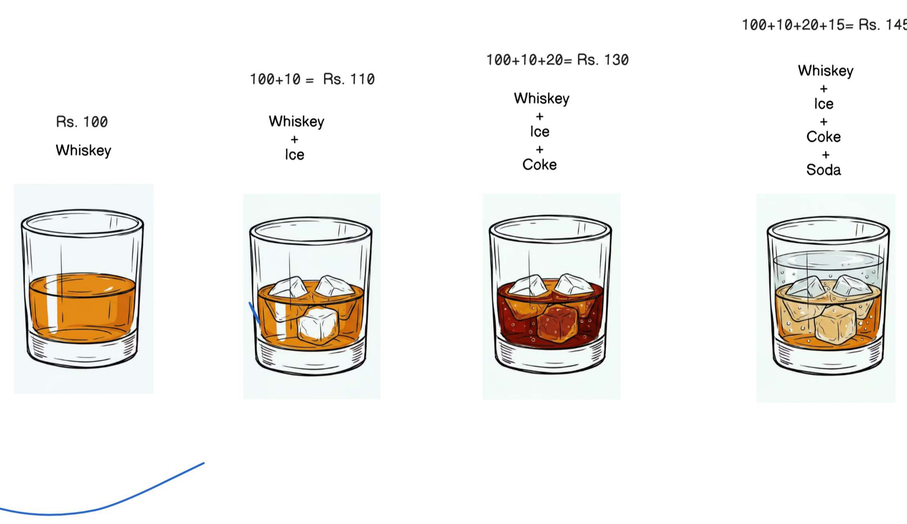



```
Drink order = new Whiskey();

order = new IceDecorator(order);

order = new StrawDecorator(order);

order.getCost();
order.setColor();


order = new CokeDecorator(order);
order = new SodaDecorator(order);
```

```
class StrawDecorator extends DrinkDecorator{

  private String color="red";

  public strawDecorator(Drink d){
    super(d);
  }

  public String getStrawColor(){
    return color;
  }

  public String setColor(String col){
    this.color = col;
  }

}
```

Problems so far:

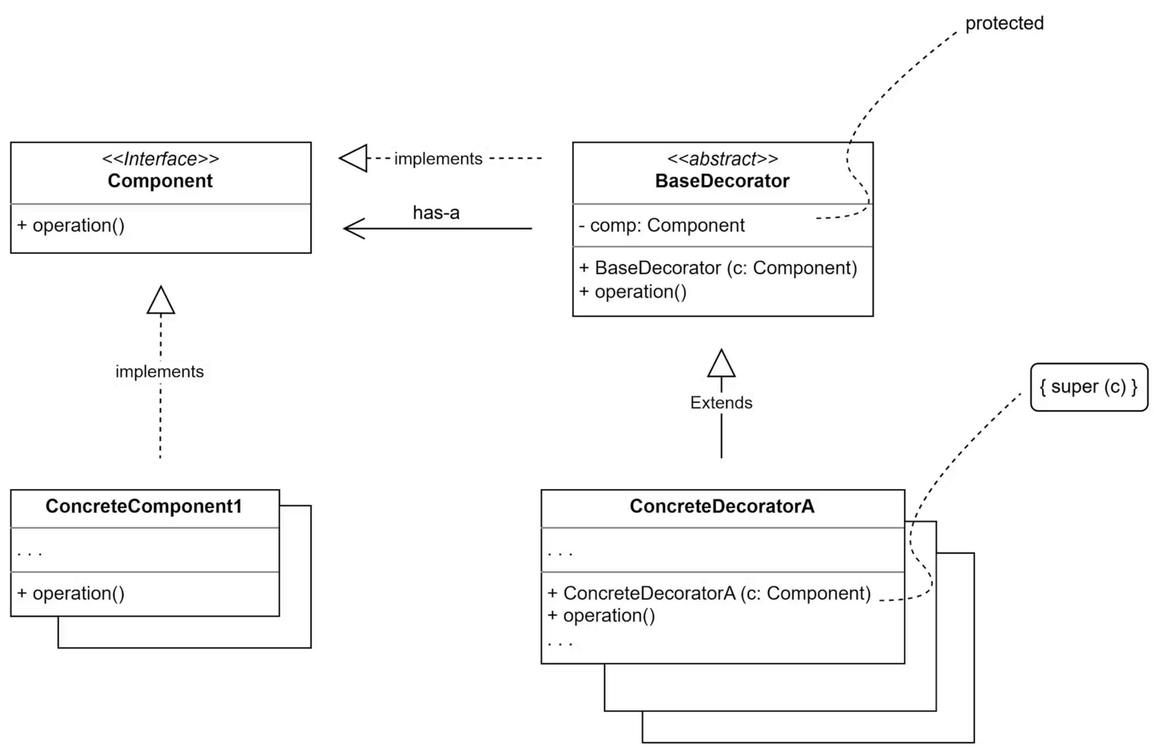
1) Compile Time decisions
2) No Multiples
3) Extension
4) Class Explosion
5) OCP (new mixer)



Not necessary that concrete decorator implements all the get cost and get description
E.g. StrawDecorator

```
class StrawDecorator extends DrinkDecorator{

  private String color="red";

  public strawDecorator(Drink d){
    super(d);
  }

  public String getStrawColor(){
    return color;
  }

  public String setColor(String col){
    this.color = col;
  }

}
```

Decorator Pattern Standard UML Diagram




```
Component c = new ConcreteComponent1();
c = new ConcreteDecoratorA(c);
c = new ConcreteDecoratorB(c);
c = new ConcreteDecoratorC(c);
```